

Alumno: Mandarin Rosales Angel Eduardo

1. El conjunto de datos MNIST se compone de imágenes numéricas escritas en cursiva del 0 al 9. Si quieres crear una red neuronal que clasifique los datos, ¿cuál es la función de activación de la última capa totalmente conectada (=densa)?

la mejor funcion de activacion para modelos con capas densas es la ReLu

2. La presión arterial, la altura y el peso forman los vectores de características. El conjunto de capacitación es el siguiente.

(121 1.72 69.0), (140 1.62 63.2), (120 1.70 59.0), (131 1.82 82.0), (101 1.78 73.5)

- (1) Suponiendo que el vector de pesos del perceptrón sea $(-0.01, 0.5, -0.23)^T$ y el sesgo es 0, explique el problema de escala con el conjunto de capacitación.

El problema de escala hace que algunas variables dominen la salida del modelo ya sea por su magnitud y no tanto por su relevancia real

- (2) Escriba el conjunto de capacitación que resulta tras la aplicación de la fórmula de preprocesamiento siguiente.

$$x_i^{new} = \frac{x_i^{old} - \mu_i}{\sigma_i} \quad (5.9)$$

- **Vector de Medias**

obteniendo el vector de la media del conjunto de datos tenemos

$$\mu = (122.6, 1.728, 69.34)$$

- **Vetor de Desviaciones estandar**

obteniendo el vector de la desviación estandar del conjunto de datos tenemos

$$\sigma = (13.03, 0.0689, 8.03)$$

- **conjuntos de datos estandarizados**

usando los resultados anteriores y aplicando la fórmula para estandarizar los datos tenemos los siguientes vectores

$$(-0.12, -0.12, -0.04) \quad (1.34, -1.57, -0.77) \quad (-0.20, -0.41, -1.29) \quad (0.64, 1.34, 1.58) \quad ($$



- (3) Explique su observación sobre si este preprocesamiento de datos alivia el problema de la escala.

La estandarización mejora la optimización del modelo ya que pone a todas las variables en la misma escala, evitando así que unas dominen más que otras por su magnitud, permitiendo que el modelo aprenda pesos que reflejen su verdadera importancia

3. En las redes neuronales, las capas de convolución se repiten varias veces en el aprendizaje profundo, lo que provoca que algunos nodos se omitan en gran medida. Qué técnica puedes utilizar para evitar este problema?

usaría la técnica de Dropout para así reducir el sobreajuste y así poder lograr una mejor generalización del modelo

5. Se desea capacitar a un clasificador cuando se tienen muchos datos de capacitación sin etiquetar, pero sólo unos pocos miles de datos etiquetados. Describe cómo puede ser útil el autoencoder y cómo trabajar.

el autoencoder no tendría problemas en trabajar con los datos ya que este realiza sus propias representaciones comprimiendo y descomprimiendo los datos para así extraer todas las características esenciales de cada imagen y así obtener una representación codificada de una dimensión menor que luego pueden utilizarse para mejorar un clasificador

```
In [7]: import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras import models, layers

from tensorflow.keras.datasets.fashion_mnist import load_data
```

```
In [8]: ## Cargamos los datos
(X_train, y_train), (X_test, y_test) = load_data()
```

```
In [9]: ## Visualización de los datos
plt.figure(figsize=(8,8))

for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(y_train[i])
    plt.axis('off')

plt.show()
```



```
In [10]: ## Realizamos el procesamiento de datos
X_train = X_train.reshape((-1,28,28,1))
X_train = X_train/255
X_test = X_test.reshape((-1,28,28,1))
X_test = X_test/255

## Definimos los parametros de capacitacion
batch_size = 8
n_epochs = 20
learn_rate = 0.0001
```

```
In [11]: model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
```

```

layers.Flatten(),
layers.Dense(64, activation='relu'),
layers.Dense(10, activation='softmax')
])

```

```

In [12]: model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=learn_rate),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_4 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_5 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_2 (Flatten)	(None, 1600)	0
dense_4 (Dense)	(None, 64)	102,464
dense_5 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)

Trainable params: 121,930 (476.29 KB)

Non-trainable params: 0 (0.00 B)

```

In [13]: history = model.fit(
    X_train, y_train,
    batch_size=batch_size,
    epochs=n_epochs,
    validation_split=0.1
)

```

Epoch 1/20
6750/6750 ————— 41s 6ms/step - accuracy: 0.7774 - loss: 0.6155 - val_
accuracy: 0.8298 - val_loss: 0.4589
Epoch 2/20
6750/6750 ————— 34s 5ms/step - accuracy: 0.8467 - loss: 0.4242 - val_
accuracy: 0.8587 - val_loss: 0.3875
Epoch 3/20
6750/6750 ————— 42s 5ms/step - accuracy: 0.8652 - loss: 0.3742 - val_
accuracy: 0.8623 - val_loss: 0.3726
Epoch 4/20
6750/6750 ————— 40s 5ms/step - accuracy: 0.8766 - loss: 0.3407 - val_
accuracy: 0.8798 - val_loss: 0.3371
Epoch 5/20
6750/6750 ————— 40s 5ms/step - accuracy: 0.8846 - loss: 0.3177 - val_
accuracy: 0.8857 - val_loss: 0.3144
Epoch 6/20
6750/6750 ————— 42s 5ms/step - accuracy: 0.8914 - loss: 0.2978 - val_
accuracy: 0.8915 - val_loss: 0.3041
Epoch 7/20
6750/6750 ————— 33s 5ms/step - accuracy: 0.8973 - loss: 0.2828 - val_
accuracy: 0.8948 - val_loss: 0.2921
Epoch 8/20
6750/6750 ————— 33s 5ms/step - accuracy: 0.9027 - loss: 0.2698 - val_
accuracy: 0.8977 - val_loss: 0.2879
Epoch 9/20
6750/6750 ————— 42s 5ms/step - accuracy: 0.9055 - loss: 0.2581 - val_
accuracy: 0.8988 - val_loss: 0.2835
Epoch 10/20
6750/6750 ————— 57s 7ms/step - accuracy: 0.9096 - loss: 0.2476 - val_
accuracy: 0.9025 - val_loss: 0.2728
Epoch 11/20
6750/6750 ————— 73s 6ms/step - accuracy: 0.9123 - loss: 0.2384 - val_
accuracy: 0.9018 - val_loss: 0.2660
Epoch 12/20
6750/6750 ————— 36s 5ms/step - accuracy: 0.9158 - loss: 0.2293 - val_
accuracy: 0.9058 - val_loss: 0.2598
Epoch 13/20
6750/6750 ————— 37s 5ms/step - accuracy: 0.9179 - loss: 0.2222 - val_
accuracy: 0.9088 - val_loss: 0.2504
Epoch 14/20
6750/6750 ————— 36s 5ms/step - accuracy: 0.9228 - loss: 0.2130 - val_
accuracy: 0.9087 - val_loss: 0.2524
Epoch 15/20
6750/6750 ————— 36s 5ms/step - accuracy: 0.9239 - loss: 0.2058 - val_
accuracy: 0.9075 - val_loss: 0.2514
Epoch 16/20
6750/6750 ————— 37s 5ms/step - accuracy: 0.9275 - loss: 0.1991 - val_
accuracy: 0.9092 - val_loss: 0.2460
Epoch 17/20
6750/6750 ————— 36s 5ms/step - accuracy: 0.9297 - loss: 0.1926 - val_
accuracy: 0.9118 - val_loss: 0.2506
Epoch 18/20
6750/6750 ————— 39s 6ms/step - accuracy: 0.9312 - loss: 0.1869 - val_
accuracy: 0.9095 - val_loss: 0.2576
Epoch 19/20
6750/6750 ————— 39s 6ms/step - accuracy: 0.9348 - loss: 0.1793 - val_

accuracy: 0.9093 - val_loss: 0.2521

Epoch 20/20

6750/6750 ————— 40s 6ms/step - accuracy: 0.9356 - loss: 0.1743 - val_
accuracy: 0.9118 - val_loss: 0.2610

```
In [14]: test_loss, test_acc = model.evaluate(X_test, y_test)
print("Accuracy en test:", test_acc)
```

```
pred = model.predict(X_test)
```

```
# clase predicha
```

```
print("Predicción:", np.argmax(pred[0]))
```

```
print("Real:", y_test[0])
```

313/313 ————— 2s 6ms/step - accuracy: 0.9024 - loss: 0.2785

Accuracy en test: 0.902400016784668

313/313 ————— 2s 5ms/step

Predicción: 9

Real: 9